



《编译原理》

# 第4讲：IR的解释执行

何冬杰  
重庆大学

# IR的解释执行

- IR仅定义程序的表示形式，本身不包含程序动态语义
- IR解释器通过执行IR，具体实现其操作语义



- 读取 IR 与输入，持续更新运行时状态，直至函数返回或程序运行出错。

# 执行过程是一系列状态转换

$$\langle pc_i, \sigma_i \rangle \rightarrow \langle pc_{i+1}, \sigma_{i+1} \rangle$$

|      | 数据状态 $\sigma$              | 控制位置 $pc$  |
|------|----------------------------|------------|
| 普通指令 | 计算新值, 修改 locals 或 memory   | 移到下一条指令    |
| 终结指令 | 通常不产生普通局部结果                | 选择后继基本块或返回 |
| 函数调用 | 建立新栈帧, 共享 globals 与 memory | 进入被调用函数入口  |

**解释器 = 状态表示 + 指令语义(含跳转控制)**

# 状态表示

## □简化的运行时环境 (Runtime Environment)

- locals映射: 记录当前函数(栈帧)中局部变量的运行时值
- symbols映射: 记录全局变量及其对应的内存地址
- 内存假设: 假定内存无限, 动态分配的内存不回收
  - memory: 记录已分配的堆(heap)和栈(stack)内存
  - stack: 记录当前函数(栈帧) 在memory中分配的栈内存, 函数退出时释放
  - region: 记录一段连续内存区域, base为起始地址, 实际data由OCaml运行时分配和管理

```
type region = { base : address; data : Bytes.t; }  
type env = { program : program; (* 当前运行程序 *)  
  symbols : (gid, address) Hashtbl.t;  
  mutable locals : (uid, int64) Hashtbl.t;  
  mutable memory : region list;  
  mutable next_addr : address; (* 下一可用内存地址 *)  
  mutable stack : region list;  
}
```

# 状态表示

## □内存分配：创建一个region

- 按8字节对齐，实际data由OCaml运行时分配和管理

```
let alloc env n =  
  if n < 0 || n > Sys.max_string_length then  
    invalid_arg "Runtime: invalid allocation size";  
  let base = env.next_addr in  
  env.next_addr <- Int64.add base (Int64.of_int (align (max 1 n) 8));  
  let region = { base; data = Bytes.make n '\000' } in  
  env.memory <- region :: env.memory;  
  region
```

- alloca调用alloc分配栈内存，记录在stack中，函数返回时清理
- alloc\_heap调用alloc分配堆内存，暂无内存回收机制

# 状态表示

## □内存存取：store和load

- 小端序(little-endian order): 一个数占多个字节时，低位字节放在低地址，高位字节放在高地址。
- 举例：把0x12345678按4个字节存到地址100:

|     |     |     |     |
|-----|-----|-----|-----|
| 78  | 56  | 34  | 12  |
| 100 | 101 | 102 | 103 |

- store: 找到地址所在region，以小端序将数据放置到指定位置

```
let store env ty v addr = let n = size ty in
  if n > 8 then invalid_arg "Runtime: aggregate store is not a scalar";
  let region, offset = find_region env addr n in
  let v = normalize ty v in
  if n = 1 then
    Bytes.set_uint8 region.data offset (Int64.to_int (Int64.logand v 255L))
  else Bytes.set_int64_le region.data offset v
```

# 状态表示

## □环境初始化: **mk\_env**

- 将所有全局符号（含 built-in 函数）存入 symbols 表
- symbols 表记录全局变量的 id 及其内存地址
- 在对应内存地址中初始化变量内容

## □全局符号存取

- symbol**和**symbol\_name**分别读取全局变量内存地址和地址上内容

## □局部变量存取: **get\_local**和**set\_local**

```
let get_local env uid =  
  match Hashtbl.find_opt env.locals uid with  
  | Some v -> v  
  | None -> failwith ("IR: unbound temporary " ^ uid)  
let set_local env uid v = Hashtbl.replace env.locals uid v
```

# 状态表示

## □函数调用：采用值传递（call-by-value）

- 保存调用者的局部环境(locals, stack), 创建被调用者的局部环境
- 函数调用结束时，恢复调用者的局部环境
- symbol和memory中非栈空间被caller和callee共享

```
let with_frame env params args f =  
  if List.length params <> List.length args  
    then failwith "Runtime: argument count mismatch";  
  let old_locals = env.locals and old_stack = env.stack in  
  env.locals <- Hashtbl.create 16; env.stack <- [];  
  Fun.protect (fun () -> List.iter2 (fun (ty, u) x ->  
    set_local env u (normalize ty x)) params args; f ())  
  ~finally:(fun () -> env.memory <- List.filter  
    (fun region -> not (List.exists ((==) region) env.stack)) env.memory;  
    env.locals <- old_locals; env.stack <- old_stack)
```

```
type cfg = block * (lbl * block) list
```

```
type block = {  
  insns : (uid * insn) list;  
  term : terminator  
}
```

# 解释执行

□本质是通过一种语言来解释并计算另一种程序的语义

- 这里，我们用Ocaml来解释LLVMLite IR程序的语义

□核心解释函数，分层解释IR语义

- exec\_func: 执行函数

- 解释执行一般从调用main函数开始，也可指定入口函数

- 需创建执行环境

```
let interpret ?(entry = "main") program =  
  exec_func (mk_env program) entry []
```

- exec\_block: 顺序执行一个基本块

- exec\_insn: 执行普通指令并更新状态

- exec\_term: 选择后继基本块或产生返回值

- eval\_bop/eval\_cmpop/eval\_casop: 计算算术/比较/类型转换运算

- eval\_opn: 将常量或局部变量转换为运行时值

# 解释执行: `exec_func`

- 提供 `print`、`println`、`alloc` 等内建函数
- 对于自定义函数，先定位函数定义，再从函数入口开始执行

```
let rec exec_func env name args =  
  match (name, args) with  
  | "print", [ v ] -> Printf.printf "%Ld" v; 0L  
  | "println", [ v ] -> Printf.printf "%Ld\n" v; 0L  
  | "alloc", [ size ] -> alloc_heap env size  
  | _ -> let func_def = match List.assoc_opt name env.program.fndefs with  
    | Some func_def -> func_def  
    | None -> failwith ("IR: unknown function " ^ name)  
  in with_frame env func_def.param args (fun () ->  
    exec_block env func_def.cfg (fst func_def.cfg))
```

# 解释执行: `exec_block, exec_term`

□ 执行块: 顺序执行一个基本块的普通指令和终结指令

```
and exec_block env cfg block =  
  List.iter (exec_insn env) block.insns;  
  exec_term env cfg block.term
```

□ 执行终结指令: 选择并执行后继基本块或产生返回值

```
and exec_term env cfg term =  
  let jump label = let blocks = snd cfg in  
    let next = List.assoc label blocks in exec_block env cfg next  
  in match term with  
  | Ret None -> 0L  
  | Ret (Some (ty, v)) -> eval_opn env ty v  
  | Br label -> jump label  
  | Cbr (cond, tlbl, flbl) -> jump (if eval_opn env l1 cond <> 0L then tlbl else flbl)
```

# 解释执行: `exec_insn`

## □ 执行普通指令并更新状态

- 通过 `set_local` 对指令计算结果值进行状态更新
- `Alloca`, `Load`, `Store` 分别分配、读取和存储内存
- 下面重点解释 `getelementptr` 指令的语义实现, 其他留作练习

```
and exec_insn env (uid, insn) =  
  let v = match insn with  
  | Alloca (elem_ty, count) -> alloca env elem_ty (eval I64 count)  
  | Load (ty, addr) -> load env ty (eval Ptr addr)  
  | Store (ty, v, addr) -> store env ty (eval ty v) (eval Ptr addr); 0L  
  | Gep (elem, base, indices) -> eval_gep env elem base indices  
  | _ -> failwith "TODO: need implementation";  
in Option.iter (fun uid -> set_local env uid v) uid
```

# Review: LLVM `getelementptr` Instruction

□ “`type insn = ... | Gep of ty * opn * typed_opn list`” in LLVM Lite

```
insn ::= ... | getelementptr t* %val, t1 idx1, t2 idx2 ,...
```

□ Example 1: 访问矩形第一个点的 x 分量

```
struct Rect { struct Point ll, lr, ul, ur };
```

```
struct Point { int x; int y; };
```

```
%tmp1 = getelementptr %Rect* %square, i32 0, i32 0
```

```
%tmp2 = getelementptr %Point* %tmp1, i32 0, i32 0
```

□ Example 2: 在 OCaml 中，数组类型可表示为指向包含长度信息的结构体的指针

```
[[int array]] = { i32, [0 x i32]}*
```

● 获取 OCaml 数组大小 `[[length a]]` :

```
[[length a]] := %1 = getelementptr {i32, [0 x i32]}* %a, i32 0, i32 0  
               %2 = load i32, i32* %1
```

# 解释执行: `exec_gep`

□聚合类型(Aggregate type): 数组或结构体类型复合而成

```
let eval_gep env elem_ty base indices =  
  let rec descend addr ty = function  
    | [] -> addr  
    | (idx_ty, idx) :: rest -> ignore (bits idx_ty);  
    match ty, idx with  
    | Array (_, item_ty), _ ->  
      let offset = Int64.mul (eval_opn env idx_ty idx) (Int64.of_int (size item_ty)) in  
      descend (Int64.add addr offset) item_ty rest  
    | Struct fields, Const _ -> let idx = eval_opn env idx_ty idx in  
      if idx < 0L || idx >= Int64.of_int (List.length fields) then  
        invalid_arg "IR: invalid struct index";  
      let i = Int64.to_int idx in  
      let addr = Int64.add addr (Int64.of_int (field_offset fields i)) in  
      descend addr (List.nth fields i) rest  
    | _ -> invalid_arg "IR: GEP needs array or constant struct index"  
  in descend (eval_opn env Ptr base) (Array (0, elem_ty)) indices
```

核心思想: 根据当前类型和索引计算基地址, 并逐层递归, 直至得到最终地址。

# 解释执行：教学展示

## □ Demo:

- Interpreter.ml, Runtime.ml, IR.ml, IRPrinter.ml, Util.ml, main.ml

## □ 编译运行:

- dune build
- dune exec ./main.exe

# 运行时错误 (Runtime Error)

□ 程序执行过程中因非法操作或异常状态导致的错误。

● 例如：数组越界、除零、空指针访问、非法内存访问等。

□ 错误处理举例：

```
let func_def = match List.assoc_opt name env.program.fndefs with
| Some func_def -> func_def
| None -> failwith ("IR: unknown function " ^ name)
in .....
```

□ 以下并非运行时错误，而是留给大家补充实现的任务：

```
| _ -> failwith "TODO: unimplemented ....."
```

# 项目1：实现LLVMLite IR解释器

- 基于简化的运行时(Runtime.ml)，实现完整的 LLVMLite IR解释器
  - 基于提供的解释器框架，补全尚未支持的LLVMLite IR语义
  - 完成后，将Interpreter.ml文件提交至OJ
  - OJ将使用多组IR程序，测试解释器的实现完整性与正确性